MINI-PROJET POO · SUJET 4

Plateforme de streaming : Oudna Play

Cahier des charges pédagogique

Wissem Zeddini · À lire avec le document « Mini-projets POO : cadre commun »

Client	M. Walid Chaouch, directeur produit de la plateforme Oudna Play (client simulé)
Le logiciel	Le cœur d'un service de streaming : un catalogue de médias (chansons, podcasts, films) et des abonnés gratuits ou premium
Difficulté	Intermédiaire (2 sur 3)
Notions du socle	Encapsulation, constructeurs, héritage, redéfinition, polymorphisme
Bonus possibles	Abstraction (badge Abstrait, avec une interface Telechargeable), exception personnalisée (badge Résilient), sauvegarde dans un fichier (badge Archiviste)
Livrable final	Code, fichier .jar, UML, rapport de 2 à 3 pages, démonstration de 5 minutes

1. Le contexte

La plateforme **Oudna Play** (un service fictif) propose des chansons, des épisodes de podcast et des films en streaming. Elle veut lancer une formule **gratuite**, financée par la publicité, à côté de sa formule **premium**. Il lui manque le cœur du logiciel : le catalogue, les lectures, les abonnés et leurs droits.

Vous êtes l'équipe de développement. Il n'y a ni son ni image : seulement des objets qui « lisent » des médias, affichés dans la console. Le temps est **simulé** : on fait « avancer » la lecture de quelques secondes, sans aucune attente réelle et sans hasard. Le client changera d'avis : concevez dès le départ pour que votre code puisse évoluer.

2. Le vocabulaire du métier

Terme	Sens pour le client
Média	Un contenu du catalogue : une chanson, un épisode de podcast ou un film.
Durée	La longueur d'un média, en secondes (un entier).
État de lecture	Arrêt, lecture ou pause.
Temps écoulé	L'endroit où en est la lecture, en secondes depuis le début.
Compteur	Le nombre de démarrages d'un média : des écoutes pour l'audio, des vues pour un film.
Playlist	Une liste de médias de tous types, avec une capacité maximale.
Abonné	Un utilisateur de la plateforme, protégé par une clé d'accès secrète.
Formule gratuite	Publicité avant chaque lecture, contenus premium interdits, quota de lectures.
Formule premium	Aucune publicité, tout le catalogue, téléchargements.
Contenu premium	Un média réservé aux abonnés premium.
Publicité	Un message montré avant la lecture à un abonné gratuit ; sa durée, en secondes, dépend du type de média.

Terme	Sens pour le client
Téléchargement	Copier un média pour l'écouter hors ligne (ici simulé : on affiche seulement un message). Seuls certains types le permettent.

3. Les règles du métier

Ces règles sont celles du client. Chacune doit être **protégée par votre code**, pas seulement respectée par vos tests.

N°	Règle	Apparaît en
R1	Le **temps écoulé** reste toujours entre 0 et la durée du média. Arrivé à la durée, le média s'arrête et le temps revient à 0.	v0.2
R2	Un média a trois **états** : arrêt, lecture, pause. On ne met en pause que pendant la lecture, on ne reprend que depuis la pause, on n'avance que pendant la lecture, et l'arrêt remet le temps à 0.	v0.2
R3	Le **compteur** augmente de 1 à chaque **démarrage** depuis l'arrêt, jamais à la reprise après une pause ; personne ne peut le modifier directement.	v0.2
R4	La **clé d'accès** d'un abonné n'est jamais lisible de l'extérieur : on peut seulement vérifier une clé proposée, et la changer en prouvant l'ancienne.	v0.2
R5	Dans une playlist, un **identifiant est unique**, et la capacité maximale ne peut pas être dépassée.	v0.3
R6	Chaque type affiche sa lecture **à sa façon** et parle de ses « écoutes » ou de ses « vues » (tableau ci-dessous).	v0.3
R7	Un abonné **gratuit** ne lit pas un contenu premium, voit une **publicité** avant chaque lecture (durée propre au type) et ne dispose que de **3 lectures**. Un abonné premium n'a ni publicité ni limite.	v1.0
R8	Seul un abonné **premium** télécharge, et seulement un média dont le type l'**autorise**.	v1.0
R9	Un **refus ne compte jamais** : ni lecture, ni publicité, ni compteur.	v1.0

Les types de médias

Type	Particularités	Compteur	Publicité (gratuit)	Téléchargeable
Média générique (classe mère)	Titre, durée	Lectures	5 secondes	Non
Chanson	Artiste, album	Écoutes	5 secondes	Oui
Épisode de podcast	Invité, sujet	Écoutes	10 secondes	Oui
Film en streaming	Réalisateur, résolution (en p)	Vues	15 secondes	Non

*Utilisez des **constantes nommées** plutôt que des nombres ou des textes écrits au milieu du code : les trois états sont des entiers constants, le quota gratuit aussi. Les durées sont des **entiers en secondes**.*

4. Version v0.1 : Cadrage

Mail du client · v0.1

De : Walid Chaouch, plateforme Oudna Play

Objet : Notre futur lecteur

Bonjour,

Nous lançons une plateforme de chansons, de podcasts et de films. Avant de coder quoi que ce soit, montrez-moi que vous avez compris mon métier : les objets que vous avez repérés, ce que chacun connaît de lui-même, et un premier média affiché à l'écran. Pas d'écran graphique : la console me suffit.

Cordialement.

N°	Demande du client	Notion
1.1	« En tant que directeur produit, je veux que vous identifiiez les objets de mon service, afin que nous parlions le même langage. »	Analyse, UML
1.2	« Je veux voir un média à l'écran, afin de vérifier que mon besoin est compris. »	Classe, constructeur, main

Vous livrez

- Un **UML de départ** d'au moins 3 classes (par exemple Media, Abonne et Demo), avec leurs attributs principaux.
- Des **squelettes de classes qui compilent**.
- Une classe **Demo** dont le main affiche un média, par exemple : `M001 · Désert d'étoiles · 3:25` (205 secondes s'écrivent 3:25).

Indice

Un média connaît au moins son **identifiant**, son **titre** et sa **durée en secondes**. Ne cherchez pas encore les trois types : ils arrivent avec le mail de la version v0.3.

5. Version v0.2 : Objets fiables

Mail du client · v0.2

De : Walid Chaouch

Objet : Des chiffres faux et une clé qui se laisse lire

Bonjour,

Merci pour votre première version. Mes testeurs ont fait des bêtises, et ça se voit : un morceau affiche **40 écoutes** alors qu'il n'a été lancé que deux fois (le compteur montait à chaque pause), on a pu mettre en pause un morceau arrêté, un titre affiche 12 minutes écoulées pour 3 minutes de durée, et pire : un testeur a pu **lire la clé d'accès** d'un abonné à l'écran !

Mes règles sont simples : le temps écoulé reste entre 0 et la durée, un média a trois états avec des passages logiques, le compteur ne compte que les vrais démarrages, et la clé d'accès ne sort jamais : on peut seulement vérifier une clé proposée.

Cordialement.

N°	Demande du client	Notion
2.1	« En tant que responsable de la fiabilité, je veux que le compteur ne change que par un démarrage de lecture, afin d'avoir des chiffres justes. » (R3)	Encapsulation
2.2	« Je veux trois états (arrêt, lecture, pause) avec des passages logiques, afin d'éviter les situations absurdes. » (R2)	État d'un objet, constantes
2.3	« Je veux faire avancer la lecture sans que le temps écoulé sorte de 0 à la durée, afin de simuler le temps. » (R1)	Méthode métier, validation
2.4	« Je veux créer un média avec son titre et sa durée, réservé ou non aux premium, afin de gagner du temps. »	Constructeurs surchargés
2.5	« Je veux que la clé d'accès d'un abonné reste secrète, se vérifie et ne se change qu'en prouvant l'ancienne, afin de protéger les comptes. » (R4)	Encapsulation, pas de getter
2.6	« Je veux lire un média en une ligne, afin de contrôler vite. »	toString()

Vous livrez

- Tous les attributs **privés**, et **aucun setter** pour le compteur, l'état, le temps écoulé et la clé d'accès.
- Deux constructeurs pour le média (avec et sans réservation premium) qui **valident** la durée, et un abonné dont la clé n'a **aucun getter**.
- Une **Demo** qui crée au moins trois objets, montre **des refus** (pause d'un média arrêté, durée invalide) et un cycle lecture, pause, reprise, arrêt.

> **Indices**
>
> Testez les **valeurs limites** : avancer de 0, de -5, d'exactement le temps restant, de plus que le temps restant. Distinguez **démarrer** (depuis l'arrêt) et **reprendre** (depuis la pause) : seul le démarrage compte. Une clé est un texte : comparez avec `equals`, jamais `==`. Un getter de la clé annulerait toute la protection.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Durée invalide (0) : 1 seconde utilisée.
M001 · Désert d'étoiles · 3:25 · lectures : 0
M002 · Parler code · 32:10 · lectures : 0 · premium
M003 · Test · 0:01 · lectures : 0
--- Lecture de M001 ---
Lecture de Désert d'étoiles
Avance de 60 s : 1:00 / 3:25
Pause de Désert d'étoiles à 1:00
Impossible d'avancer : Désert d'étoiles n'est pas en lecture.
Reprise de Désert d'étoiles à 1:00
Fin de Désert d'étoiles
Impossible de mettre en pause : Désert d'étoiles n'est pas en lecture.
Lecture de Désert d'étoiles
Arrêt de Désert d'étoiles
M001 · Désert d'étoiles · 3:25 · lectures : 2
--- Clé d'accès ---
Abonné Sara · gratuit
Clé 0000 correcte ? false
Clé sara42 correcte ? true
Changement refusé : la nouvelle clé doit avoir au moins 4 caractères.
Nouvelle clé correcte ? true
```

6. Version v0.3 : La famille

Mail du client · v0.3

De : Walid Chaouch

Objet : Trois sortes de médias, et ce n'est pas fini

Bonjour,

Bravo : le compteur est enfin juste et mes clés sont à l'abri. Passons au vrai catalogue ! J'ai trois sortes de médias : des chansons, des épisodes de podcast et des films. Chacun a ses particularités (voir le tableau), s'affiche à sa façon quand on le lit, et parle de ses « écoutes » ou de ses « vues ».

Je veux des playlists, tous types mélangés, que je puisse afficher et lire d'un coup, avec leur durée totale.

Et une chose importante : bientôt viendront les livres audio, les directs et d'autres formats. **Je ne veux pas que vous réécriviez la playlist à chaque nouveau format.**

Cordialement.

N°	Demande du client	Notion
3.1	« Je veux trois types de médias avec leurs particularités (artiste, invité, résolution…), afin de décrire fidèlement mon catalogue. »	Héritage, super(...)
3.2	« Je veux que chaque type affiche sa lecture à sa façon et ait son vocabulaire de compteur, afin d'être fidèle à chaque format. » (R6)	Redéfinition, @Override
3.3	« Je veux une playlist qui contient des médias de tous types, avec une capacité maximale et sans doublon d'identifiant, afin de refléter mes listes. » (R5)	Tableau ou liste du type commun
3.4	« Je veux afficher toute la playlist en une fois, afin de la contrôler. »	Une seule boucle, toString()
3.5	« Je veux lire toute la playlist et connaître sa durée totale, afin de proposer des soirées complètes. »	Polymorphisme
3.6	« Je veux ajouter un nouveau format plus tard sans modifier la playlist, afin que mon logiciel évolue. »	Preuve du polymorphisme

Vous livrez

- Une **classe mère concrète** qui porte ce qui est commun (identifiant, titre, durée, état, compteur), et **trois classes filles** qui n'ajoutent que leurs particularités. L'UML est mis à jour.
- Une **Playlist** qui contient des médias de types différents, et **une seule boucle** pour les afficher, les lire et calculer la durée totale.
- La preuve que la playlist n'a pas besoin d'être modifiée pour accueillir un nouveau format.

Indices

Si vous écrivez **instanceof** dans la boucle de la playlist, c'est que le polymorphisme manque : quelle méthode chaque type devrait-il redéfinir ? Quand une fille redéfinit `lire()`, elle doit appeler `super.lire()` : sans cet appel, l'état et le compteur de la classe mère ne bougent plus. Pour comparer deux identifiants, utilisez `equals`.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Identifiant S101 déjà présent : ajout refusé.
Playlist pleine (4/4) : ajout refusé.
Playlist Soirée (4/4)
  Chanson S101 · Désert d'étoiles · 3:25 · écoutes : 0
  Podcast P201 · Parler code · 32:10 · écoutes : 0
  Film F301 · Horizon Sud · 1:52:00 · vues : 0 · premium
  M001 · Jingle · 0:12 · lectures : 0
Durée totale : 2:27:47
Lecture de la playlist Soirée
Lecture de Désert d'étoiles · Nour Ayari · album Sable
  Fin de Désert d'étoiles
Lecture de Parler code · invité Hela Dridi · sujet Cloud
  Fin de Parler code
Lecture de Horizon Sud · de Leila Jlassi · 1080p
  Fin de Horizon Sud
Lecture de Jingle
  Fin de Jingle
Playlist Soirée (4/4)
  Chanson S101 · Désert d'étoiles · 3:25 · écoutes : 1
  Podcast P201 · Parler code · 32:10 · écoutes : 1
  Film F301 · Horizon Sud · 1:52:00 · vues : 1 · premium
  M001 · Jingle · 0:12 · lectures : 1
```

7. Version v1.0 : Livraison

Mail du client · v1.0

De : Walid Chaouch

Objet : Dernière étape avant la formule gratuite

Bonjour,

La playlist fonctionne bien et mes équipes sont ravies. Dernière étape avant le lancement de la **formule gratuite**, financée par la publicité. Mes règles : l'abonné gratuit ne lit pas les contenus réservés aux premium, voit une publicité avant chaque lecture (plus longue pour un épisode que pour une chanson) et n'a droit qu'à 3 lectures. L'abonné premium n'a ni publicité ni limite, et peut télécharger, mais seulement les médias qui s'y prêtent : pas les films. Et un refus ne doit **rien compter** : ni publicité vue, ni lecture, ni écoute.

Mon ancien lecteur contenait un gros bloc de « si c'est un film… sinon si c'est une chanson… » pour décider de la publicité et des téléchargements, qu'il fallait rouvrir à chaque nouveau format. Je ne veux plus de ça.

Un menu console pour mes équipes, et un fichier que je puisse lancer moi-même.

Cordialement.

N°	Demande du client	Notion
4.1	« Je veux qu'un abonné lise un média selon les droits de sa formule, afin de respecter mes offres. » (R7)	Défi métier, assemblage
4.2	« Je veux une publicité avant chaque lecture d'un abonné gratuit, de durée propre au type de média, afin de financer la formule gratuite. » (R7)	Redéfinition, polymorphisme
4.3	« Je veux que seuls les premium téléchargent, et seulement les médias téléchargeables, sans que l'abonné demande le type du média, afin que mes nouveaux formats s'intègrent seuls. » (R8)	Défi métier, polymorphisme
4.4	« Je veux qu'un refus ne compte pour rien, afin d'avoir des statistiques justes. » (R9)	Ordre des vérifications
4.5	« Je veux un menu console, afin que mes équipes utilisent le logiciel sans toucher au code. »	Menu, boucle de saisie

N°	Demande du client	Notion
4.6	« Je veux un fichier que je lance moi-même, afin de l'installer sur mon poste. »	Fichier .jar

Le défi métier : gratuit ou premium

Quand un abonné demande une lecture, le logiciel vérifie **dans cet ordre** :

1. le média est-il déjà en cours (lecture ou pause) ? Refus ;

2. le média est-il réservé aux premium alors que l'abonné est gratuit ? Refus ;

3. l'abonné gratuit a-t-il épuisé ses 3 lectures ? Refus ;

4. si l'abonné est gratuit, la **publicité** passe (durée propre au type) et s'ajoute à son bilan ;

5. le média démarre : son compteur et le nombre de lectures de l'abonné augmentent de 1.

Un **refus ne compte jamais**. Le **téléchargement** (simulé) n'est accepté que pour un abonné premium et un média téléchargeable.

L'abonné **ne demande jamais** « quel est ton type ? » au média : c'est le média qui sait s'il se télécharge et combien dure sa publicité.

Menu console attendu (exemple) :

```
=== OUDNA PLAY ===
1. Afficher le catalogue
2. Se connecter
3. Lire un média
4. Contrôler une lecture
5. Télécharger un média
6. Passer premium
7. Voir mon bilan
0. Quitter
Votre choix (0 à 7) :
```

Exemple de sortie d'une Demo de formules (la vôtre peut différer) :

```
--- Abonné Sara · gratuit ---
Publicité : 5 secondes avant Désert d'étoiles.
Lecture de Désert d'étoiles · Nour Ayari · album Sable
Accès refusé : Horizon Sud est réservé aux abonnés premium.
Publicité : 10 secondes avant Parler code.
Lecture de Parler code · invité Hela Dridi · sujet Cloud
Publicité : 5 secondes avant Désert d'étoiles.
Lecture de Désert d'étoiles · Nour Ayari · album Sable
Quota atteint : 3 lectures gratuites utilisées. Passez premium pour continuer.
Téléchargement refusé : réservé aux abonnés premium.
--- Abonné Karim · premium ---
Lecture de Horizon Sud · de Leila Jlassi · 1080p
Téléchargement impossible : Horizon Sud n'est pas téléchargeable.
Téléchargement simulé : Parler code (32:10).
--- Sara passe premium ---
Lecture de Parler code · invité Hela Dridi · sujet Cloud
Téléchargement simulé : Désert d'étoiles (3:25).
--- Bilans ---
Sara · premium · lectures 4 · publicité 20 s · téléchargements 1
Karim · premium · lectures 1 · publicité 0 s · téléchargements 1
Chanson S101 · Désert d'étoiles · 3:25 · écoutes : 2
Film F301 · Horizon Sud · 1:52:00 · vues : 1 · premium
```

Indices

La **tentation** : un grand `if` avec **instanceof**, ou un cast comme `(FilmStream) media`, pour choisir la publicité ou interdire le téléchargement. C'est exactement le bloc que le client refuse : chaque nouveau format obligerait à rouvrir l'abonné. La bonne voie : des méthodes que chaque type redéfinit, comme `estTelechargeable()` et `getDureePublicite()`.

Lisez bien l'**ordre** des vérifications : augmenter le quota ou la publicité **avant** d'avoir vérifié l'accès compte un refus. Passer premium doit changer le comportement dès la lecture suivante, sans autre modification.

8. Les bonus du sujet

Badge	Défi
Abstrait	Créez une **interface** `Telechargeable` (méthode `telecharger`) implémentée par les seuls types qui s'y prêtent, et supprimez la méthode booléenne du socle. Où l'abonné doit-il alors vérifier qu'un média est téléchargeable ? Comparez les deux conceptions dans le rapport et justifiez votre choix (une classe mère abstraite est une autre voie : que répond Java à `new Media(...)` ?).
Résilient	Créez une **exception personnalisée** (par exemple `AccesRefuseException`) levée par une lecture refusée, et interceptée par le menu qui affiche un message clair.
Archiviste	Sauvegardez le catalogue (avec ses compteurs) et les abonnés dans un fichier texte ou CSV à la fermeture, et rechargez-les au démarrage. Que faites-vous de la **clé d'accès** ? L'écrire en clair dans un fichier annule la protection : justifiez votre choix dans le rapport.

9. Les pièges fréquents de ce sujet

- **Un instanceof ou un cast pour décider de la publicité ou du téléchargement** : le polymorphisme est contourné ; un nouveau format obligerait à modifier l'abonné. C'est le piège principal de ce sujet.
- **Un getter pour la clé d'accès** : il annule la protection. Seule la vérification doit exister.
- **Un compteur qui augmente à la reprise après une pause**, ou à une lecture refusée.
- **Redéfinir lire() sans appeler super.lire()** : l'état et le compteur de la classe mère ne sont plus mis à jour.
- **Un temps écoulé qui dépasse la durée** ou devient négatif : testez avancer(0), avancer(-5) et une avance trop grande.
- **Des passages d'état absurdes** : mettre en pause un média arrêté, avancer pendant une pause.
- **Compter la publicité ou le quota avant d'avoir vérifié l'accès** : un refus ne doit rien compter.
- **Des états écrits en texte libre** (« lecture », « Lecture », « play ») : utilisez des constantes.
- **Comparer des chaînes (clé, identifiant) avec ==**, ou **oublier super(...)** dans le constructeur d'une fille.
- **Recopier la gestion de l'état dans chaque fille** : si vous copiez, c'est que la classe mère manque.

10. Pour votre rapport

En plus de la structure du cadre commun, répondez à ces questions propres au sujet :

- Pourquoi la clé d'accès n'a-t-elle pas de getter ? Que permet la vérification à la place, et quelle règle est ainsi protégée ?
- Dans les classes filles, qu'apporte l'appel de `super.lire()` ? Que se passerait-il sans lui ?

- Quelle est la ligne de code où le **type réel** d'un média décide de la publicité ou du téléchargement ?
- Que faudrait-il modifier pour ajouter les livres audio ? Combien de classes existantes ?

11. Modifications en direct possibles à la soutenance

Pendant la démonstration, on pourra vous demander l'une de ces modifications, en quelques minutes :

- Ajouter un **livre audio** (auteur, narrateur ; téléchargeable ; publicité de 8 secondes) et le lire depuis une playlist **sans toucher à la playlist ni à l'abonné**.
- Passer le quota gratuit de 3 à 5 lectures.
- Rendre les films téléchargeables.
- Passer la publicité d'un épisode de podcast de 10 à 20 secondes.
- Afficher dans le bilan les minutes de publicité vues, au lieu des secondes.

Annexe : les liens entre les classes (indicatif)

Voici seulement les **relations** attendues à l'arrivée. Les attributs et les méthodes, c'est à vous de les concevoir.

Relation	Sens
Chanson, EpisodePodcast, FilmStream → Media	Héritage : chaque type **est un** média.
Playlist → Media (de 0 à plusieurs)	La playlist **contient** des médias de tous types, par le type commun.
Abonne → Media	L'abonné **utilise** un média (lecture, téléchargement) sans connaître son type.
Demo, Menu → Playlist, Abonne	Les classes de test et de menu **utilisent** la playlist et l'abonné.